

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (Tiếng Việt)

Aurélien Géron — bản dịch chỉnh sửa (2nd ed.)

Ngày 6 tháng 7 năm 2026

Mục lục

1	Lời tựa	2
1.1	Sóng thần Machine Learning	2
1.2	Machine Learning trong dự án của bạn	2
1.3	Mục tiêu và cách tiếp cận	2
1.4	Điều kiện tiên quyết	3
1.5	Lộ trình	3
1.6	Những thay đổi trong ấn bản thứ hai	4
1.7	Tài nguyên khác	5
1.8	Các quy ước được sử dụng trong cuốn sách này	6
1.9	Ví dụ về code	6
1.10	Sử dụng ví dụ về code	6
1.11	Học trực tuyến O'Reilly	7
1.12	Cách liên hệ với chúng tôi	7
1.13	Lời cảm ơn	7
2	Phần I Các nguyên tắc cơ bản của Machine Learning	9
2.1	Chương 1 Phong cảnh Machine Learning	9
2.1.1	Machine Learning là gì?	10
2.1.2	Tại sao nên sử dụng Machine Learning?	10
2.1.3	Ví dụ về ứng dụng	11
2.1.4	Các loại hệ thống Machine Learning	12
2.1.4.1	Supervised/Unsupervised Learning	13
2.1.4.2	Batch Learning và Online Learning	16
2.1.4.3	Instance-Based Learning so với Model-Based Learning	17

1 Lời tựa

1.1 Sóng thần Machine Learning

Năm 2006, Geoffrey Hinton và cộng sự. Đã xuất bản một bài báo hướng dẫn cách training một deep neural network có khả năng nhận dạng các chữ số viết tay với độ chính xác tiên tiến nhất (>98%). Họ đặt tên cho kỹ thuật này là “Deep Learning”. Deep neural network là một model (rất) đơn giản hóa của vỏ não của chúng ta, bao gồm một chồng các lớp tế bào thần kinh nhân tạo. Việc training một mạng lưới thần kinh sâu được nhiều người coi là không thể vào thời điểm đó và hầu hết các nhà nghiên cứu đã từ bỏ ý tưởng này vào cuối những năm 1990. Bài viết này đã thu hút sự quan tâm của cộng đồng khoa học và không lâu sau, nhiều bài báo mới đã chứng minh rằng Deep Learning không chỉ khả thi mà còn có khả năng đạt được những thành tựu đáng kinh ngạc mà không kỹ thuật Machine Learning (ML) nào khác có thể hy vọng so sánh được (với sự trợ giúp của sức mạnh tính toán cực lớn và lượng dữ liệu khổng lồ). Sự nhiệt tình này nhanh chóng mở rộng sang nhiều lĩnh vực khác của Machine Learning.

Khoảng một thập kỷ sau, Machine Learning đã chinh phục ngành công nghiệp này: nó là trung tâm của phần lớn điều kỳ diệu trong các sản phẩm công nghệ cao ngày nay, xếp hạng kết quả tìm kiếm trên web của bạn, tăng cường khả năng nhận dạng giọng nói cho điện thoại thông minh của bạn, đề xuất video và đánh bại nhà vô địch thế giới trong trò chơi cờ vây. Trước khi bạn biết điều đó, nó sẽ lái xe của bạn.

1.2 Machine Learning trong dự án của bạn

Vì vậy, đương nhiên là bạn rất hào hứng với Machine Learning và rất muốn tham gia bữa tiệc!

Có lẽ bạn muốn cung cấp cho robot tự chế của mình một bộ não riêng? Làm cho nó nhận diện khuôn mặt? Hoặc học cách đi bộ xung quanh?

Hoặc có thể công ty của bạn có rất nhiều dữ liệu (nhật ký user, dữ liệu tài chính, dữ liệu sản xuất, dữ liệu cảm biến máy, số liệu thống kê đường dây nóng, báo cáo HR, v.v.) và nhiều khả năng bạn có thể khai quật được một số viên ngọc ẩn nếu bạn biết tìm ở đâu. Với Machine Learning, bạn có thể thực hiện được những điều sau và hơn thế nữa:

- Phân khúc khách hàng và tìm ra chiến lược tiếp thị tốt nhất cho từng nhóm.
- Đề xuất sản phẩm cho từng khách hàng dựa trên những gì khách hàng tương tự đã mua.
- Phát hiện những giao dịch có khả năng gian lận.
- Dự báo doanh thu năm tới.

Dù lý do là gì đi nữa, bạn đã quyết định tìm hiểu Machine Learning và áp dụng nó trong các dự án của mình. Ý tưởng tuyệt vời!

1.3 Mục tiêu và cách tiếp cận

Cuốn sách này giả định rằng bạn gần như không biết gì về Machine Learning. Mục tiêu của nó là cung cấp cho bạn các khái niệm, công cụ và trực giác mà bạn cần để implement các chương trình có khả năng học hỏi từ dữ liệu.

Chúng ta sẽ đề cập đến một số lượng lớn các kỹ thuật, từ đơn giản nhất và được sử dụng phổ biến nhất (chẳng hạn như Linear Regression) đến một số kỹ thuật Deep Learning thường xuyên giành chiến thắng trong các cuộc thi.

Thay vì tự implement các phiên bản đồ chơi của từng thuật toán, chúng ta sẽ sử dụng các framework Python sẵn sàng trong sản xuất:

- Scikit-Learn rất dễ sử dụng mà nó còn implement nhiều thuật toán Machine Learning một cách hiệu quả, vì vậy nó trở thành điểm khởi đầu tuyệt vời cho việc học Machine Learning. Nó được David Cournapeau tạo ra vào năm 2007 và hiện được dẫn dắt bởi một nhóm các nhà nghiên cứu tại Viện Nghiên cứu Khoa học Máy tính và Tự động hóa Pháp (Inria).

- TensorFlow là một thư viện phức tạp hơn để tính toán số phân tán. Nó giúp có thể training và chạy neural networks rất lớn một cách hiệu quả bằng cách phân phối các tính toán trên hàng trăm máy chủ đa GPU (đơn vị xử lý đồ họa). TensorFlow (TF) được tạo tại Google và hỗ trợ nhiều ứng dụng Machine Learning quy mô lớn của hãng. Nó có nguồn mở vào tháng 11 năm 2015 và phiên bản 2.0 được phát hành vào tháng 9 năm 2019.

- Keras là Deep Learning API cấp cao giúp việc training và chạy neural networks rất đơn giản. Nó có thể chạy trên TensorFlow, Theano hoặc Microsoft Cognitive Toolkit (trước đây gọi là CNTK). TensorFlow đi kèm với implementation riêng của API này, gọi là tf.keras, cung cấp hỗ trợ cho một số TensorFlow features nâng cao (ví dụ: khả năng load dữ liệu hiệu quả).

Cuốn sách ủng hộ cách tiếp cận hands-on, nâng cao sự hiểu biết trực quan về Machine Learning thông qua các ví dụ hoạt động cụ thể và chỉ một chút lý thuyết. Mặc dù bạn có thể đọc cuốn sách này mà không cần lấy máy tính xách tay của mình ra, nhưng tôi thực sự khuyên bạn nên thử nghiệm các ví dụ code có sẵn trực tuyến dưới dạng Jupyter notebooks tại <https://github.com/ageron/handson-ml2>.

1.4 Điều kiện tiên quyết

Cuốn sách này giả định rằng bạn có một số kinh nghiệm lập trình Python và bạn đã quen thuộc với các thư viện khoa học chính của Python—đặc biệt là NumPy, pandas và Matplotlib.

Ngoài ra, nếu bạn quan tâm đến những gì sâu xa, bạn cũng nên có hiểu biết hợp lý về toán cấp đại học (giải tích, đại số tuyến tính, xác suất và thống kê).

Nếu bạn chưa biết Python thì <http://learnpython.org/> là một nơi tuyệt vời để bắt đầu. Hướng dẫn chính thức trên Python.org cũng khá hay.

Nếu bạn chưa bao giờ sử dụng Jupyter, Chương 2 sẽ hướng dẫn bạn cài đặt và những điều cơ bản: đây là một công cụ mạnh mẽ cần có trong hộp công cụ của bạn.

Nếu bạn không quen với các thư viện khoa học của Python, Jupyter notebooks được cung cấp sẽ bao gồm một số hướng dẫn. Ngoài ra còn có hướng dẫn toán nhanh về đại số tuyến tính.

1.5 Lộ trình

Cuốn sách này được tổ chức thành hai phần. Phần I, Nguyên tắc cơ bản của Machine Learning, bao gồm các chủ đề sau:

- Machine Learning là gì, nó cố gắng giải quyết những vấn đề gì, các danh mục chính và khái niệm cơ bản về hệ thống của nó
- Các bước trong một dự án Machine Learning điển hình
- Học bằng cách khớp model với dữ liệu
- Tối ưu hóa hàm chi phí

- Xử lý, làm sạch và chuẩn bị dữ liệu
- Lựa chọn và thiết kế features
- Chọn model và điều chỉnh hyperparameters bằng cross-validation
- Những thách thức của Machine Learning, đặc biệt là underfitting và overfitting (sự đánh đổi bias/variance)
 - Các thuật toán học phổ biến nhất: Linear và Polynomial Regression, Logistic Regression, k-Nearest Neighbors, Support Vector Machines, Decision Trees, Random Forests và Ensemble methods
 - Giảm chiều của dữ liệu training để chống lại “lời nguyền về chiều”
 - Các kỹ thuật unsupervised learning khác, bao gồm clustering, ước tính mật độ và phát hiện dị thường

Phần II, Neural Networks và Deep Learning, bao gồm các chủ đề sau:

- Mạng lưới thần kinh là gì và chúng có tác dụng gì
- Xây dựng và training mạng lưới thần kinh sử dụng TensorFlow và Keras
- Kiến trúc neural network quan trọng nhất: neural network chuyển tiếp cho dữ liệu dạng bảng, mạng tích chập cho thị giác máy tính, mạng regression và mạng bộ nhớ ngắn hạn dài (LSTM) để xử lý chuỗi, bộ mã hóa/giải mã và Bộ chuyển đổi cho natural language processing, autoencoder và mạng đối nghịch tổng quát (GANs) để học tổng quát
 - Kỹ thuật training mạng lưới thần kinh sâu
 - Cách xây dựng một agent (ví dụ: bot trong trò chơi) có thể học các chiến lược tốt thông qua thử và sai, sử dụng Reinforcement Learning
 - Load và xử lý trước lượng lớn dữ liệu một cách hiệu quả
 - Training và deploying TensorFlow models trên quy mô lớn

Phần đầu tiên chủ yếu dựa trên Scikit-Learn, trong khi phần thứ hai sử dụng TensorFlow và Keras.

Đừng nhảy xuống vùng nước sâu quá vội vàng: mặc dù Deep Learning chắc chắn là một trong những lĩnh vực thú vị nhất trong Machine Learning, nhưng trước tiên bạn nên nắm vững các nguyên tắc cơ bản. Hơn nữa, hầu hết các bài toán đều có thể được giải quyết khá tốt bằng cách sử dụng các kỹ thuật đơn giản hơn như Random Forests và Ensemble methods (được thảo luận ở Phần I). Deep Learning phù hợp nhất cho các vấn đề phức tạp như nhận dạng hình ảnh, nhận dạng giọng nói hoặc natural language processing, miễn là bạn có đủ dữ liệu, khả năng tính toán và sự kiên nhẫn.

1.6 Những thay đổi trong ấn bản thứ hai

Phiên bản thứ hai này có sáu mục tiêu chính:

1. Bao gồm các chủ đề ML bổ sung: nhiều kỹ thuật unsupervised learning hơn (bao gồm clustering, phát hiện dị thường, ước tính mật độ và hỗn hợp models); nhiều kỹ thuật hơn để training mạng sâu (bao gồm cả mạng tự chuẩn hóa); các kỹ thuật thị giác máy tính bổ sung (bao gồm Xception, SNet, phát hiện đối tượng bằng YOLO và phân đoạn ngữ nghĩa bằng R-CNN); xử lý các chuỗi bằng cách sử dụng neural networks convolutional (CNNs, bao gồm WaveNet); xử lý ngôn ngữ tự nhiên bằng cách sử dụng neural networks (RNNs), CNNs và Trans-forms định kỳ; và GANs.
2. Bao gồm các thư viện bổ sung và APIs (Keras, Data API, TF-Agent for Reinforcement Learning) cũng như training và triển khai TF models trên quy mô lớn bằng cách sử dụng Chiến lược phân phối API, TF-Serving và Nền tảng Google Cloud AI. Đồng thời giới thiệu ngắn gọn về TF Transform, TFLite, TF Addons/Seq2Seq và TensorFlow.js.
3. Thảo luận một số kết quả quan trọng mới nhất từ nghiên cứu Deep Learning.

4. Di chuyển tất cả các chương TensorFlow sang TensorFlow 2 và sử dụng implementation Keras API trên TensorFlow (tf.keras) bất cứ khi nào có thể.

5. Cập nhật các mẫu code để sử dụng các phiên bản mới nhất của Scikit-Learn, NumPy, pandas, Matplotlib và các thư viện khác.

6. Làm rõ một số phần và sửa một số lỗi, nhờ có nhiều phản hồi tốt từ độc giả.

Một số chương đã được thêm vào, một số khác được viết lại và một số được sắp xếp lại. Xem <https://homl.info/changes2> để biết thêm chi tiết về những gì đã thay đổi trong phiên bản thứ hai.

1.7 Tài nguyên khác

Có nhiều tài nguyên tuyệt vời để tìm hiểu về Machine Learning. Ví dụ: khóa học ML của Andrew Ng trên Coursera thật tuyệt vời, mặc dù nó đòi hỏi đầu tư thời gian đáng kể (hàng tháng).

Ngoài ra còn có nhiều trang web thú vị về Machine Learning, bao gồm cả Hướng dẫn sử dụng đặc biệt của Scikit-Learn. Bạn cũng có thể thưởng thức Dataquest, nơi cung cấp các hướng dẫn tương tác rất hay và các blog ML chẳng hạn như các blog được liệt kê trên Quora. Cuối cùng, trang web Deep Learning có một danh sách tài nguyên hữu ích để bạn khám phá và tìm hiểu thêm.

Còn rất nhiều sách giới thiệu khác về Machine Learning. Đặc biệt:

- Khoa học dữ liệu từ đầu của Joel Grus (O'Reilly) trình bày các nguyên tắc cơ bản của Machine Learning và triển khai một số thuật toán chính trong Python thuần túy (từ đầu, như tên cho thấy).

- Machine Learning của Stephen Marsland: Phối cảnh thuật toán (Chapman & Hall) là phần giới thiệu tuyệt vời về Machine Learning, bao gồm nhiều chủ đề chuyên sâu với các ví dụ code trong Python (cũng từ đầu, nhưng sử dụng NumPy).

- Python Machine Learning (Packt Publishing) của Sebastian Raschka cũng là phần giới thiệu tuyệt vời về Machine Learning và tận dụng các thư viện nguồn mở Python (Pylearn 2 và Theano).

- Deep Learning with Python (Manning) của François Chollet là một cuốn sách rất thực tế bao gồm nhiều chủ đề một cách rõ ràng và ngắn gọn, như bạn có thể mong đợi từ tác giả của thư viện Keras xuất sắc. Nó thiên về các ví dụ về code hơn là lý thuyết toán học.

- Cuốn sách Một trăm trang Machine Learning của Andriy Burkov rất ngắn và bao gồm nhiều chủ đề ấn tượng, giới thiệu chúng bằng những thuật ngữ dễ tiếp cận mà không né tránh các phương trình toán học.

- Học từ dữ liệu (AMLBook) của Yaser S. Abu-mostafa, Malik Magdon-Ismael và Hsuan-Tien Lin là một cách tiếp cận mang tính lý thuyết đối với ML, cung cấp những hiểu biết sâu sắc, đặc biệt là về sự đánh đổi giữa bias/variance (xem Chương 4).

- Trí tuệ nhân tạo của Stuart Russell và Peter Norvig: Phương pháp tiếp cận hiện đại, tái bản lần thứ 3 (Pearson), là một cuốn sách hay (và khổng lồ) bao gồm vô số chủ đề đáng kinh ngạc, bao gồm cả Machine Learning. Nó giúp đưa ML vào quan điểm.

Cuối cùng, việc tham gia các trang web cạnh tranh ML như Kaggle.com sẽ cho phép bạn lab các kỹ năng của mình về các vấn đề trong thế giới thực, với sự trợ giúp và hiểu biết sâu sắc từ một số chuyên gia ML giỏi nhất hiện có.

1.8 Các quy ước được sử dụng trong cuốn sách này

Các quy ước đánh máy sau đây được sử dụng trong cuốn sách này:

Chữ nghiêng: Cho biết các thuật ngữ mới, URLs, địa chỉ email, tên tệp và phần mở rộng tệp.

Độ rộng không đổi: Được sử dụng cho danh sách chương trình, cũng như trong các đoạn văn để đề cập đến các thành phần chương trình như tên biến hoặc hàm, cơ sở dữ liệu, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa.

In đậm có chiều rộng không đổi: Hiển thị các lệnh hoặc văn bản khác mà user phải nhập theo nghĩa đen.

Nghiêng có độ rộng không đổi: Hiển thị văn bản cần được thay thế bằng các giá trị do user cung cấp hoặc bằng các giá trị được xác định theo ngữ cảnh.

1.9 Ví dụ về code

Có một loạt Jupyter notebooks chứa đầy tài liệu bổ sung, chẳng hạn như ví dụ về code và bài tập, có sẵn để tải xuống tại <https://github.com/ageron/handsonml2>.

Một số ví dụ về code trong sách loại bỏ các phần hoặc chi tiết lặp đi lặp lại rõ ràng hoặc không liên quan đến Machine Learning. Điều này giúp tập trung vào các phần quan trọng của code và tiết kiệm không gian để đề cập đến nhiều chủ đề hơn. Nếu bạn muốn có ví dụ về code đầy đủ, tất cả chúng đều có sẵn trong Jupyter notebooks.

Lưu ý rằng khi các ví dụ code hiển thị một số đầu ra, các ví dụ code này được hiển thị với lời nhắc Python (`>` và `...`), như trong shell Python, để phân biệt rõ ràng code với các đầu ra. Ví dụ, đoạn code này định nghĩa hàm `Square()`, sau đó nó tính toán và hiển thị bình phương của 3:

```
> def square(x): ... return x ** 2 ... > result = square(3) > result 9
```

Khi code không hiển thị bất cứ điều gì, lời nhắc sẽ không được sử dụng. Tuy nhiên, kết quả đôi khi có thể được hiển thị dưới dạng comment, như thế này:

```
def square(x): return x ** 2 result = square(3) # result is 9
```

1.10 Sử dụng ví dụ về code

Cuốn sách này ra đời để giúp bạn hoàn thành công việc của mình. Nói chung, nếu code ví dụ được cung cấp kèm theo cuốn sách này, bạn có thể sử dụng nó trong các chương trình và tài liệu của mình. Bạn không cần liên hệ với chúng tôi để xin phép trừ khi bạn đang sao chép một phần đáng kể của code. Ví dụ: viết một chương trình sử dụng nhiều đoạn code từ cuốn sách này không cần phải có sự cho phép. Việc bán hoặc phân phối ví dụ CD-ROM từ sách của O'Reilly cần phải có sự cho phép. Trả lời câu hỏi bằng cách trích dẫn cuốn sách này và trích dẫn code ví dụ không cần phải có sự cho phép. Việc kết hợp một lượng lớn code ví dụ từ cuốn sách này vào tài liệu sản phẩm của bạn cần phải có sự cho phép.

Chúng tôi đánh giá cao nhưng không yêu cầu ghi công. Ghi công thường bao gồm tiêu đề, tác giả, nhà xuất bản và ISBN. Ví dụ: "Hands-On Machine Learning với Scikit-Learn, Keras và TensorFlow, Phiên bản thứ 2, của Aurélien Géron (O'Reilly). Bản quyền 2019 Kiwisoft S.A.S., 978-1-492-03264-9." Nếu bạn cảm thấy việc sử dụng các ví dụ về code của mình nằm ngoài phạm vi sử dụng hợp pháp hoặc sự cho phép nêu trên, vui lòng liên hệ với chúng tôi theo địa chỉ Permission@oreilly.com.

1.11 Học trực tuyến O'Reilly

Trong gần 40 năm, O'Reilly Media đã cung cấp training, kiến thức và hiểu biết sâu sắc về công nghệ và kinh doanh để giúp các công ty thành công.

Mạng lưới chuyên gia và nhà đổi mới độc đáo của chúng tôi chia sẻ kiến thức và chuyên môn của họ thông qua sách, bài báo, hội nghị và nền tảng học tập trực tuyến của chúng tôi. Nền tảng học tập trực tuyến của O'Reilly cung cấp cho bạn quyền truy cập theo yêu cầu vào các khóa training trực tiếp, lộ trình học tập chuyên sâu, môi trường mã hóa tương tác cũng như bộ sưu tập văn bản và video khổng lồ từ O'Reilly và hơn 200 nhà xuất bản khác. Để biết thêm thông tin, vui lòng truy cập <http://oreilly.com>.

1.12 Cách liên hệ với chúng tôi

Vui lòng giải quyết các nhận xét và câu hỏi liên quan đến cuốn sách này tới nhà xuất bản:

O'Reilly Media, Inc. 1005 Gravenstein Highway North Sebastopol, CA 95472 800-998-9938 (ở Hoa Kỳ hoặc Canada) 707-829-0515 (quốc tế hoặc địa phương) 707-829-0104 (fax)

Chúng tôi có một trang web dành cho cuốn sách này, nơi chúng tôi liệt kê các lỗi sai, ví dụ và bất kỳ thông tin bổ sung nào. Bạn có thể truy cập trang này tại <https://homl.info/oreilly2>.

Để bình luận hoặc đặt câu hỏi kỹ thuật về cuốn sách này, hãy gửi email đến bookquestions@oreilly.com.

Để biết thêm thông tin về sách, khóa học, hội nghị và tin tức của chúng tôi, hãy xem trang web của chúng tôi tại <http://www.oreilly.com>.

Tìm chúng tôi trên Facebook: <http://facebook.com/oreilly>

Theo dõi chúng tôi trên Twitter: <http://twitter.com/oreillymedia>

Xem chúng tôi trên YouTube: <http://www.youtube.com/oreillymedia>

1.13 Lời cảm ơn

Chưa bao giờ trong những giấc mơ điên rồ nhất của mình, tôi lại tưởng tượng rằng ấn bản đầu tiên của cuốn sách này sẽ thu hút được lượng độc giả lớn như vậy. Tôi đã nhận được rất nhiều tin nhắn từ độc giả, nhiều câu hỏi, một số vui lòng chỉ ra lỗi sai và phần lớn gửi cho tôi những lời động viên. Tôi không thể bày tỏ lòng biết ơn của mình đối với tất cả những độc giả này vì sự hỗ trợ to lớn của họ. Cảm ơn tất cả các bạn rất nhiều! Vui lòng gửi các vấn đề trên GitHub nếu bạn tìm thấy lỗi trong các ví dụ code (hoặc chỉ để đặt câu hỏi) hoặc gửi lỗi nếu bạn tìm thấy lỗi trong văn bản. Một số độc giả cũng chia sẻ cuốn sách này đã giúp họ có được công việc đầu tiên như thế nào hoặc nó đã giúp họ giải quyết một vấn đề cụ thể mà họ đang giải quyết như thế nào. Tôi thấy những phản hồi như vậy có động lực vô cùng lớn. Nếu bạn thấy cuốn sách này hữu ích, tôi sẽ rất vui nếu bạn có thể chia sẻ câu chuyện của mình với tôi, một cách riêng tư (ví dụ: qua LinkedIn) hoặc công khai (ví dụ: trong một tweet hoặc thông qua bài đánh giá trên Amazon).

Tôi cũng vô cùng biết ơn tất cả những người tuyệt vời đã dành thời gian trong cuộc sống bận rộn của họ để xem lại cuốn sách của tôi một cách cẩn thận như vậy. Đặc biệt, tôi muốn cảm ơn François Chollet vì đã xem xét tất cả các chương dựa trên Keras và TensorFlow và đã cho tôi một số phản hồi sâu sắc. Vì Keras là một trong những phần bổ

sung chính cho ấn bản thứ hai này nên việc được tác giả đánh giá cuốn sách là vô giá. Tôi thực sự khuyên bạn nên

Cuốn sách Deep Learning với Python (Manning) của François: nó có sự ngắn gọn, rõ ràng và sâu sắc của chính thư viện Keras. Tôi cũng xin đặc biệt cảm ơn Ankur Patel, người đã xem xét từng chương của ấn bản thứ hai này và cho tôi những phản hồi tuyệt vời, đặc biệt là về Chương 9, bao gồm các kỹ thuật unsupervised learning. Anh ấy có thể viết cả một cuốn sách về chủ đề này... ồ, chờ đã, anh ấy đã viết được! Hãy xem Lab Unsupervised Learning bằng Python: Cách xây dựng các giải pháp Machine Learning ứng dụng từ dữ liệu không được gán nhãn (O'Reilly). Xin gửi lời cảm ơn sâu sắc tới Olzhas Akpambetov, người đã xem lại tất cả các chương trong phần thứ hai của cuốn sách, đã kiểm tra phần lớn code và đưa ra nhiều gợi ý tuyệt vời. Tôi biết ơn Mark Daoust, Jon Krohn, Dominic Monn và Josh Patterson vì đã xem lại phần thứ hai của cuốn sách này một cách kỹ lưỡng và đưa ra những kiến thức chuyên môn của họ. Họ không ngần ngại lật tẩy và đưa ra những phản hồi vô cùng hữu ích.

Khi viết ấn bản thứ hai này, tôi đã may mắn nhận được nhiều sự giúp đỡ từ các thành viên của nhóm TensorFlow—đặc biệt là Martin Wicke, người đã trả lời không một mỗi hàng tá câu hỏi của tôi và gửi những câu hỏi còn lại đến đúng người, bao gồm Karmel Allison, Paige Bailey, Eugene Brevdo, William Chargin, Daniel “Wolff” Dobson, Nick Felt, Bruce Fontaine, Goldie Gadde, Sandeep Gupta, Priya Gupta, Kevin Haas, Konstantinos Katsiapis, Viacheslav Kovalevskiy, Allen Lavoie, Clemens Mewald, Dan Moldovan, Sean Morgan, Tom O'Malley, Alexandre Passos, André Susano Pinto, Anthony Platanios, Oscar Ramirez, Anna Revinskaya, Saurabh Saxena, Ryan Sepassi, Jiri Simsa, Xiaodan Song, Christina Sorokin, Dustin Tran, Todd Wang, Pete Warden (người cũng đã đánh giá phần đầu tiên) ấn bản) Edd Wilder-James và Yuefeng Zhou, tất cả đều rất hữu ích. Xin chân thành cảm ơn tất cả các bạn và tất cả các thành viên khác của nhóm TensorFlow, không chỉ vì sự giúp đỡ mà còn vì đã tạo ra một thư viện tuyệt vời như vậy! Đặc biệt cảm ơn Irene Giannoumis và Robert Crowe của nhóm TFX vì đã xem xét sâu Chương 13 và 19.

Tôi cũng xin gửi lời cảm ơn chân thành tới đội ngũ nhân viên tuyệt vời của O'Reilly, đặc biệt là Nicole Taché, người đã cho tôi những phản hồi sâu sắc và luôn vui vẻ, khích lệ và hữu ích: Tôi không thể mơ về một biên tập viên giỏi hơn. Xin chân thành cảm ơn Michele Cronin, người rất hữu ích (và kiên nhẫn) khi bắt đầu ấn bản thứ hai này, và Kristen Brown, biên tập viên sản xuất cho ấn bản thứ hai, người đã xem xét tất cả các bước (cô ấy cũng điều phối các bản sửa lỗi và cập nhật cho mỗi lần tái bản của ấn bản đầu tiên). Tôi cũng xin gửi lời cảm ơn đến Rachel Monaghan và Amanda Kersey vì đã biên tập kỹ lưỡng (lần lượt cho ấn bản thứ nhất và thứ hai), cũng như tới Johnny O'Toole, người đã quản lý mối quan hệ với Amazon và trả lời nhiều câu hỏi của tôi. Cảm ơn Marie Beaugureau, Ben Lorica, Mike Loukides và Laurel Ruma vì đã tin tưởng vào dự án này và giúp tôi xác định phạm vi của nó. Cảm ơn Matt Hacker và tất cả nhóm Atlas đã trả lời tất cả các câu hỏi kỹ thuật của tôi về định dạng, AsciiDoc và LaTeX, đồng thời cảm ơn Nick Adams, Rebecca Demest, Rachel Head, Judith McConville, Helen Monroe, Karen Montgomery, Rachel Roumeliotis và mọi người khác tại O'Reilly đã đóng góp cho cuốn sách này.

Tôi cũng muốn cảm ơn các đồng nghiệp cũ ở Google, đặc biệt là nhóm YouTube video classification, vì đã dạy tôi rất nhiều điều về Machine Learning. Tôi không bao giờ có thể bắt đầu ấn bản đầu tiên nếu không có họ. Đặc biệt xin gửi lời cảm ơn đặc biệt đến các chuyên gia ML của cá nhân tôi: Clément Courbet, Julien Dubois, Mathias Kende, Daniel Kitachewsky, James Pack, Alexander Pak, Anosh Raj, Vitor Sessak, Wiktor Tomczak, Ingrid von Glehn và Rich Washington. Và xin cảm ơn tất cả những người khác mà tôi đã

làm việc cùng tại YouTube cũng như trong các nhóm nghiên cứu tuyệt vời của Google ở Mountain View. Xin gửi lời cảm ơn chân thành đến Martin Andrews, Sam Witteveen và Jason Zaman vì đã chào đón tôi gia nhập nhóm Chuyên gia nhà phát triển Google của họ ở Singapore, với sự hỗ trợ tử tế của Soonson Kwon và vì tất cả những cuộc thảo luận tuyệt vời mà chúng tôi đã có về Deep Learning và TensorFlow. Bất kỳ ai quan tâm đến Deep Learning ở Singapore chắc chắn nên tham gia buổi gặp mặt Deep Learning Singapore của họ. Jason xứng đáng được cảm ơn đặc biệt vì đã chia sẻ một số kiến thức chuyên môn về TFLite của anh ấy cho Chương 19!

Tôi sẽ không bao giờ quên những người tốt bụng đã đánh giá ấn bản đầu tiên của cuốn sách này, bao gồm David Andrzejewski, Lukas Biewald, Justin Francis, Vincent Guilbeau, Eddy Hung, Karim Matrah, Grégoire Mesnil, Salim Sémaoune, Iain Smears, Michel Tessier, Ingrid von Glehn, Pete Warden, và tất nhiên là người anh trai thân yêu của tôi Sylvain. Tôi đặc biệt cảm ơn Haesun Park, người đã cho tôi rất nhiều phản hồi xuất sắc và đã mắc một số lỗi khi viết bản dịch tiếng Hàn cho ấn bản đầu tiên của cuốn sách này. Anh ấy cũng đã dịch Jupyter notebooks sang tiếng Hàn, chưa kể đến tài liệu của TensorFlow. Tôi không nói được tiếng Hàn, nhưng xét theo chất lượng phản hồi của anh ấy thì tất cả các bản dịch của anh ấy hẳn phải thực sự xuất sắc! Haesun cũng vui lòng đóng góp một số lời giải cho các bài tập trong ấn bản thứ hai này.

Cuối cùng nhưng không kém phần quan trọng, tôi vô cùng biết ơn người vợ yêu dấu của tôi, Emmanuelle, và ba đứa con tuyệt vời của chúng tôi, Alexandre, Rémi và Gabrielle, vì đã khuyến khích tôi làm việc chăm chỉ cho cuốn sách này. Tôi cũng biết ơn họ vì sự tò mò vô độ của họ: việc giải thích một số khái niệm khó nhất trong cuốn sách này cho vợ con tôi đã giúp tôi làm sáng tỏ suy nghĩ của mình và trực tiếp cải thiện nhiều phần trong đó. Và họ tiếp tục mang cho tôi bánh quy và cà phê! Người ta có thể mơ ước gì hơn nữa?

2 Phần I Các nguyên tắc cơ bản của Machine Learning

2.1 Chương 1 Phong cảnh Machine Learning

Khi hầu hết mọi người nghe “Machine Learning”, họ hình dung ra một con robot: một quần gia đáng tin cậy hoặc một Kẻ hủy diệt chết người, tùy thuộc vào người bạn hỏi. Nhưng Machine Learning không chỉ là một ảo mộng tương lai; nó đã ở đây rồi. Trên thực tế, nó đã tồn tại hàng thập kỷ trong một số ứng dụng chuyên biệt, chẳng hạn như Nhận dạng ký tự quang học (OCR). Nhưng ứng dụng ML đầu tiên thực sự trở thành xu hướng chủ đạo, cải thiện cuộc sống của hàng trăm triệu người, đã chiếm lĩnh thế giới vào những năm 1990: bộ lọc thư rác. Nó không hẳn là một Skynet tự nhận biết, nhưng về mặt kỹ thuật nó đủ tiêu chuẩn là Machine Learning (nó thực sự đã học rất tốt nên bạn hiếm khi cần gắn cờ email là thư rác nữa). Tiếp theo là hàng trăm ứng dụng ML hiện đang âm thầm cung cấp năng lượng cho hàng trăm sản phẩm và features mà bạn sử dụng thường xuyên, từ những đề xuất tốt hơn cho đến tìm kiếm bằng giọng nói.

Machine Learning bắt đầu từ đâu và kết thúc ở đâu? Chính xác thì việc một cỗ máy học được điều gì đó có ý nghĩa gì? Nếu tôi tải xuống một bản sao Wikipedia, máy tính của tôi có thực sự học được điều gì không? Nó đột nhiên thông minh hơn? Trong chương này, chúng ta sẽ bắt đầu bằng cách làm rõ Machine Learning là gì và tại sao bạn có thể muốn sử dụng nó.

Sau đó, trước khi bắt đầu khám phá lục địa Machine Learning, chúng ta sẽ xem bản đồ và tìm hiểu về các khu vực chính và các địa danh đáng chú ý nhất: supervised learning so với unsupervised learning, online learning so với batch learning, instancebased learning

so với model-based learning. Sau đó, chúng ta sẽ xem xét quy trình làm việc của một dự án ML điển hình, thảo luận về những thách thức chính mà bạn có thể gặp phải và đề cập đến cách đánh giá và tuning hệ thống Machine Learning.

Chương này giới thiệu rất nhiều khái niệm cơ bản (và biệt ngữ) mà mọi nhà khoa học dữ liệu nên thuộc lòng. Đây sẽ là một phần tổng quan cấp cao (đây là chương duy nhất không có nhiều code), tất cả đều khá đơn giản, nhưng bạn nên đảm bảo mọi thứ đều rõ ràng trước khi tiếp tục phần còn lại của cuốn sách. Vì vậy, hãy lấy một ly cà phê và bắt đầu!

Nếu bạn đã biết tất cả những điều cơ bản về Machine Learning, bạn có thể chuyển thẳng đến Chương 2. Nếu bạn không chắc chắn, hãy thử trả lời tất cả các câu hỏi được liệt kê ở cuối chương trước khi chuyển sang phần khác.

2.1.1 Machine Learning là gì?

Machine Learning là khoa học (và nghệ thuật) lập trình máy tính để chúng có thể học hỏi từ dữ liệu.

Đây là một định nghĩa tổng quát hơn một chút:

[Machine Learning là] lĩnh vực nghiên cứu cung cấp cho máy tính khả năng học hỏi mà không cần lập trình rõ ràng.

—Arthur Samuel, 1959

Và một định hướng kỹ thuật hơn:

Một chương trình máy tính được cho là học từ kinh nghiệm E đối với một số nhiệm vụ T và một số thước đo hiệu suất P , nếu hiệu suất của nó trên T , được đo bằng P , cải thiện theo kinh nghiệm E .

—Tom Mitchell, 1997

Bộ lọc thư rác của bạn là một chương trình Machine Learning, dựa trên các ví dụ về email spam (ví dụ: bị user gắn cờ) và ví dụ về các email thông thường (không phải thư rác, còn được gọi là “ham”), có thể học cách gắn cờ thư rác. Các ví dụ mà hệ thống sử dụng để học được gọi là tập training. Mỗi training example được gọi là một phiên bản training (hoặc mẫu). Trong trường hợp này, nhiệm vụ T là gắn cờ thư rác cho các email mới, trải nghiệm E là dữ liệu training và cần xác định thước đo hiệu suất P ; ví dụ: bạn có thể sử dụng tỷ lệ email được phân loại chính xác. Thước đo hiệu suất cụ thể này được gọi là độ chính xác và nó thường được sử dụng trong các tác vụ classification.

Nếu bạn chỉ tải xuống một bản sao của Wikipedia, máy tính của bạn có nhiều dữ liệu hơn nhưng lại không đột nhiên tốt hơn ở bất kỳ tác vụ nào. Vì vậy, việc tải xuống một bản sao của Wikipedia không phải là Machine Learning.

2.1.2 Tại sao nên sử dụng Machine Learning?

Hãy xem xét cách bạn viết bộ lọc thư rác bằng các kỹ thuật lập trình truyền thống:

1. Trước tiên, bạn hãy xem thư rác thường trông như thế nào. Bạn có thể nhận thấy rằng một số từ hoặc cụm từ (chẳng hạn như “4U”, “thể tín dụng”, “miễn phí” và “tuyệt vời”) có xu hướng xuất hiện nhiều trong dòng chủ đề. Có lẽ bạn cũng sẽ nhận thấy một số mẫu khác trong tên người gửi, nội dung email và các phần khác của email.

2. Bạn sẽ viết thuật toán phát hiện cho từng mẫu mà bạn nhận thấy và chương trình của bạn sẽ gắn cờ email là thư rác nếu phát hiện thấy một số mẫu này.

3. Bạn sẽ kiểm tra chương trình của mình và lặp lại bước 1 và 2 cho đến khi chương trình đủ tốt để khởi chạy.

Vì vấn đề rất khó nên chương trình của bạn có thể sẽ trở thành một danh sách dài các quy tắc phức tạp—khá khó để duy trì.

Ngược lại, bộ lọc thư rác dựa trên kỹ thuật Machine Learning sẽ tự động tìm hiểu những từ và cụm từ nào là predictors tốt của thư rác bằng cách phát hiện các mẫu từ thường gặp bất thường trong các ví dụ thư rác so với các ví dụ ham. Chương trình ngắn hơn nhiều, dễ bảo trì hơn và rất có thể chính xác hơn.

Điều gì sẽ xảy ra nếu những kẻ gửi thư rác nhận thấy rằng tất cả email có chứa “4U” của họ đều bị chặn? Thay vào đó, họ có thể bắt đầu viết “Dành cho bạn”. Bộ lọc thư rác sử dụng các kỹ thuật lập trình truyền thống sẽ cần được cập nhật để gắn cờ các email “Dành cho bạn”. Nếu những kẻ gửi thư rác tiếp tục làm việc xung quanh bộ lọc thư rác của bạn, bạn sẽ cần phải viết các quy tắc mới mãi mãi.

Ngược lại, bộ lọc thư rác dựa trên kỹ thuật Machine Learning sẽ tự động nhận thấy rằng “Dành cho U” đã trở nên thường xuyên bất thường trong các thư rác bị user gắn cờ và nó bắt đầu gắn cờ chúng mà không cần sự can thiệp của bạn.

Một lĩnh vực khác mà Machine Learning tỏa sáng là giải quyết các vấn đề quá phức tạp đối với các phương pháp tiếp cận truyền thống hoặc không có thuật toán nào được biết đến. Ví dụ, hãy xem xét nhận dạng giọng nói. Giả sử bạn muốn bắt đầu đơn giản và viết một chương trình có khả năng phân biệt các từ “một” và “hai”. Bạn có thể nhận thấy rằng từ “hai” bắt đầu bằng âm thanh có âm vực cao (“T”), vì vậy bạn có thể mã hóa cứng một thuật toán đo cường độ âm thanh có âm vực cao và sử dụng thuật toán đó để phân biệt một và hai—nhưng rõ ràng kỹ thuật này sẽ không mở rộng tới hàng nghìn từ được nói bởi hàng triệu người rất khác nhau trong môi trường ồn ào và trong hàng chục ngôn ngữ. Giải pháp tốt nhất (ít nhất là ở thời điểm hiện tại) là viết một thuật toán có khả năng tự học, đưa ra nhiều bản ghi ví dụ cho mỗi từ.

Cuối cùng, Machine Learning có thể giúp con người học hỏi. Các thuật toán ML có thể được kiểm tra để xem chúng đã học được những gì (mặc dù đối với một số thuật toán, điều này có thể phức tạp). Ví dụ: khi bộ lọc thư rác đã được training về đủ thư rác, nó có thể dễ dàng được kiểm tra để tiết lộ danh sách các từ và tổ hợp từ mà nó tin là predictors tốt nhất trong thư rác. Đôi khi điều này sẽ tiết lộ những điều không ngờ tới.

Mối tương quan hoặc xu hướng mới, và do đó dẫn đến sự hiểu biết tốt hơn về vấn đề. Việc áp dụng các kỹ thuật ML để khai thác lượng lớn dữ liệu có thể giúp khám phá các mẫu chưa rõ ràng ngay lập tức. Điều này được gọi là khai thác dữ liệu.

Tóm lại, Machine Learning rất phù hợp cho:

- Các vấn đề mà giải pháp hiện tại đòi hỏi phải tuning nhiều hoặc danh sách quy tắc dài: một thuật toán Machine Learning thường có thể đơn giản hóa code và hoạt động tốt hơn so với phương pháp truyền thống.
- Các vấn đề phức tạp mà việc sử dụng phương pháp truyền thống không mang lại giải pháp tốt: các kỹ thuật Machine Learning tốt nhất có thể tìm ra giải pháp.
- Môi trường biến động: hệ thống Machine Learning có thể thích ứng với dữ liệu mới.
- Nhận thông tin chi tiết về các vấn đề phức tạp và lượng dữ liệu lớn.

2.1.3 Ví dụ về ứng dụng

Hãy xem xét một số ví dụ cụ thể về nhiệm vụ Machine Learning, cùng với các kỹ thuật có thể giải quyết chúng:

Phân tích hình ảnh của các sản phẩm trên dây chuyền sản xuất để tự động classification. Đây là hình ảnh classification, thường được thực hiện bằng cách sử dụng mạng nơ ron tích chập (CNNs; xem Chương 14).

Phát hiện khối u trong quá trình quét não Đây là phân đoạn ngữ nghĩa, trong đó mỗi pixel trong hình ảnh được phân loại (vì chúng ta muốn xác định chính xác vị trí và hình dạng của khối u), thường sử dụng CNNs.

Tự động classification các bài báo Đây là natural language processing (NLP), và cụ thể hơn là classification văn bản, có thể được giải quyết bằng cách sử dụng neural networks (RNNs), CNNs hoặc Transformers lặp lại (xem Chương 16).

Tự động gắn cờ các bình luận xúc phạm trên các diễn đàn thảo luận Đây cũng là văn bản classification, sử dụng cùng công cụ NLP.

Tự động tóm tắt các tài liệu dài Đây là một nhánh của NLP được gọi là tóm tắt văn bản, một lần nữa sử dụng các công cụ tương tự.

Tạo chatbot hoặc trợ lý cá nhân Việc này liên quan đến nhiều thành phần NLP, bao gồm hiểu ngôn ngữ tự nhiên (NLU) và mô-đun trả lời câu hỏi.

Dự báo doanh thu của công ty bạn vào năm tới, dựa trên nhiều chỉ số hiệu suất. Đây là nhiệm vụ regression (tức là dự đoán các giá trị) có thể được giải quyết bằng cách sử dụng bất kỳ regression model nào, chẳng hạn như Linear Regression hoặc Polynomial Regression model (xem Chương 4), regression SVM (xem Chương 5), regression Random Forest (xem Chương 7) hoặc neural network nhân tạo (xem Chương 10). Nếu bạn muốn tính đến chuỗi số liệu hiệu suất trong quá khứ, bạn có thể muốn sử dụng RNNs, CNNs hoặc Transformers (xem Chương 15 và 16).

Làm cho ứng dụng của bạn phản ứng với lệnh thoại Đây là tính năng nhận dạng giọng nói, yêu cầu xử lý các mẫu âm thanh: vì chúng là các chuỗi dài và phức tạp nên chúng thường được xử lý bằng RNNs, CNNs hoặc Transformers (xem Chương 15 và 16).

Phát hiện gian lận thẻ tín dụng Đây là phát hiện bất thường (xem Chương 9).

Phân khúc khách hàng dựa trên giao dịch mua hàng của họ để bạn có thể thiết kế chiến lược tiếp thị khác nhau cho từng phân khúc. Đây là clustering (xem Chương 9).

Biểu diễn dataset phức tạp, có nhiều chiều trong một sơ đồ rõ ràng và sâu sắc. Đây là cách trực quan hóa dữ liệu, thường liên quan đến các kỹ thuật giảm kích thước (xem Chương 8).

Đề xuất một sản phẩm mà khách hàng có thể quan tâm dựa trên những lần mua hàng trước đây. Đây là một hệ thống gợi ý. Một cách tiếp cận là cung cấp các giao dịch mua trước đây (và thông tin khác về khách hàng) vào neural network nhân tạo (xem Chương-

Ter 10) và làm cho nó xuất ra lần mua hàng tiếp theo có nhiều khả năng xảy ra nhất. Mạng lưới thần kinh này thường sẽ được training về các chuỗi mua hàng trước đây của tất cả khách hàng.

Xây dựng bot thông minh cho trò chơi Vấn đề này thường được giải quyết bằng cách sử dụng Reinforcement Learning (RL; xem Chương 18), là một nhánh của Machine Learning training các đặc vụ (chẳng hạn như bot) để chọn các hành động sẽ tối đa hóa phần thưởng của họ theo thời gian (ví dụ: bot có thể nhận được phần thưởng mỗi khi người chơi mất một số điểm sinh mệnh), trong một môi trường nhất định (chẳng hạn như trò chơi). Chương trình AlphaGo nổi tiếng đánh bại nhà vô địch thế giới trong trò chơi cờ vây được xây dựng bằng RL.

Danh sách này có thể còn dài, nhưng hy vọng nó sẽ mang lại cho bạn cảm giác về mức độ rộng và phức tạp đáng kinh ngạc của các nhiệm vụ mà Machine Learning có thể giải quyết cũng như các loại kỹ thuật mà bạn sẽ sử dụng cho từng nhiệm vụ.

2.1.4 Các loại hệ thống Machine Learning

Có rất nhiều loại hệ thống Machine Learning khác nhau nên rất hữu ích khi classification chúng theo các danh mục rộng, dựa trên các tiêu chí sau:

- Họ có được training với sự giám sát của con người hay không (supervised, unsupervised, semisupervised và Reinforcement Learning)

- Liệu họ có thể học dần dần một cách nhanh chóng hay không (online learning so với batch learning)

- Cho dù chúng hoạt động bằng cách đơn giản so sánh các điểm dữ liệu mới với các điểm dữ liệu đã biết hay thay vào đó bằng cách phát hiện các mẫu trong dữ liệu training và xây dựng model mang tính dự đoán, giống như các nhà khoa học vẫn làm (instance-based learning so với model-based learning)

Những tiêu chí này không độc quyền; bạn có thể kết hợp chúng theo bất kỳ cách nào bạn muốn. Ví dụ: bộ lọc thư rác hiện đại có thể học nhanh chóng bằng cách sử dụng mạng thần kinh sâu model được training bằng cách sử dụng các ví dụ về thư rác và ham; điều này làm cho nó trở thành một online, modelbased, supervised learning system.

Chúng ta hãy xem xét từng tiêu chí này kỹ hơn một chút.

2.1.4.1 Supervised/Unsupervised Learning

Hệ thống Machine Learning có thể được phân loại theo số lượng và loại hình giám sát mà chúng nhận được trong quá trình training. Có bốn loại chính: supervised learning, unsupervised learning, semisupervised learning và Reinforcement Learning.

Supervised learning

Trong supervised learning, training set mà bạn cung cấp cho thuật toán bao gồm các giải pháp mong muốn, được gọi là label.

Nhiệm vụ supervised learning điển hình là classification. Bộ lọc thư rác là một ví dụ điển hình về điều này: nó được training với nhiều email mẫu cùng với loại của chúng (thư rác hoặc ham) và nó phải học cách classification email mới.

Một nhiệm vụ điển hình khác là dự đoán giá trị số mục tiêu, chẳng hạn như giá một chiếc ô tô, dựa trên một bộ features (số km đã đi, tuổi, nhãn hiệu, v.v.) có tên là predictors. Loại nhiệm vụ này được gọi là regression. Để training hệ thống, bạn cần cho nó nhiều example về ô tô, bao gồm cả predictors và label của chúng (tức là giá của chúng).

Trong Machine Learning, thuộc tính là một loại dữ liệu (ví dụ: “số dặm”), trong khi feature có một số ý nghĩa, tùy thuộc vào ngữ cảnh, nhưng nhìn chung có nghĩa là một thuộc tính cộng với giá trị của nó (ví dụ: “số dặm = 15.000”). Nhiều người sử dụng attribute và feature thay thế cho nhau.

Lưu ý rằng một số thuật toán regression cũng có thể được sử dụng cho classification và ngược lại. Ví dụ: Logistic Regression thường được sử dụng cho classification, vì nó có thể xuất ra một giá trị tương ứng với xác suất thuộc về một lớp nhất định (ví dụ: 20% khả năng là thư rác).

Sự thật thú vị: cái tên nghe có vẻ kỳ quặc này là một thuật ngữ thống kê được Francis Galton đưa ra khi ông đang nghiên cứu thực tế rằng con cái của những người cao thường có xu hướng thấp hơn cha mẹ chúng. Vì những đứa trẻ thấp hơn nên anh ấy gọi nó là regression. Tên này sau đó được áp dụng cho các phương pháp mà ông sử dụng để phân tích mối tương quan giữa các biến số.

Dưới đây là một số thuật toán supervised learning quan trọng nhất (được đề cập trong cuốn sách này):

- k-Nearest Neighbors
- Linear Regression
- Logistic Regression
- Support Vector Machines (SVMs)

- Decision Trees và Random Forests
- Neural networks

Một số kiến trúc neural network có thể không supervised, chẳng hạn như autoencoder và máy Boltzmann bị hạn chế. Chúng cũng có thể được bán giám sát, chẳng hạn như trong mạng lưới niềm tin sâu sắc và training trước không supervised.

Unsupervised learning

Trong unsupervised learning, như bạn có thể đoán, dữ liệu training không được gắn label. Hệ thống cố gắng học mà không có giáo viên.

Dưới đây là một số thuật toán unsupervised learning quan trọng nhất (hầu hết chúng được đề cập trong Chương 8 và 9):

- Clustering
 - K-Means
 - DBSCAN
 - Phân tích cụm phân cấp (HCA)
- Phát hiện sự bất thường và phát hiện tính mới
 - SVM một lớp
 - Rừng Cách Ly
- Trực quan hóa và giảm kích thước
 - Principal Component Analysis (PCA)
 - Hạt nhân PCA
 - Nhúng tuyến tính cục bộ (LLE)
 - Nhúng hàng xóm ngẫu nhiên phân tán t-t-Distributed (t-SNE)
- Học quy tắc kết hợp
 - Apriori
 - Eclat

Ví dụ: giả sử bạn có nhiều dữ liệu về khách truy cập blog của mình. Bạn có thể muốn chạy thuật toán clustering để cố gắng phát hiện các nhóm khách truy cập tương tự. Bạn không bao giờ có thể cho thuật toán biết khách truy cập thuộc nhóm nào: nó tìm thấy những kết nối đó mà không cần sự trợ giúp của bạn. Ví dụ: có thể nhận thấy rằng 40% khách truy cập là nam giới yêu thích truyện tranh và thường đọc blog của bạn vào buổi tối, trong khi 20% là những người trẻ tuổi yêu thích khoa học viễn tưởng ghé thăm vào cuối tuần. Nếu bạn sử dụng thuật toán clustering phân cấp, thuật toán này cũng có thể chia mỗi nhóm thành các nhóm nhỏ hơn. Điều này có thể giúp bạn nhắm mục tiêu bài viết của mình cho từng nhóm.

Các thuật toán trực quan hóa cũng là ví dụ điển hình về thuật toán unsupervised learning: bạn cung cấp cho chúng rất nhiều dữ liệu phức tạp và không được gắn nhãn, đồng thời chúng tạo ra biểu diễn 2D hoặc 3D cho dữ liệu của bạn để có thể dễ dàng vẽ đồ thị. Các thuật toán này cố gắng duy trì nhiều cấu trúc nhất có thể (ví dụ: cố gắng giữ các cụm riêng biệt trong không gian đầu vào không bị chồng chéo trong hình ảnh trực quan) để bạn có thể hiểu cách sắp xếp dữ liệu và có thể xác định các mẫu không bị nghi ngờ.

Hãy chú ý xem động vật ở khá xa khỏi xe cộ và ngựa ở gần hươu nhưng lại ở xa chim. Hình được sao chép lại với sự cho phép của Richard Socher và cộng sự, “Học tập không cần chụp qua chuyển giao CrossModal,” Kỷ yếu của Hội nghị quốc tế lần thứ 26 về Hệ thống xử lý thông tin thần kinh 1 (2013): 935–943.

Một nhiệm vụ liên quan là giảm kích thước, trong đó mục tiêu là đơn giản hóa dữ liệu mà không làm mất quá nhiều thông tin. Một cách để làm điều này là hợp nhất một số

features tương ứng thành một. Ví dụ: quãng đường đã đi của một chiếc ô tô có thể tương quan chặt chẽ với tuổi của nó, vì vậy thuật toán giảm kích thước sẽ hợp nhất chúng thành một feature đại diện cho độ hao mòn của ô tô. Điều này được gọi là trích xuất feature.

Thông thường, bạn nên cố gắng giảm kích thước của dữ liệu training bằng thuật toán giảm kích thước trước khi đưa nó vào một thuật toán Machine Learning khác (chẳng hạn như thuật toán học có giám sát). Nó sẽ chạy nhanh hơn nhiều, dữ liệu sẽ chiếm ít dung lượng ổ đĩa và bộ nhớ hơn và trong một số trường hợp, nó cũng có thể hoạt động tốt hơn.

Tuy nhiên, một nhiệm vụ không supervised quan trọng khác là phát hiện sự bất thường—ví dụ: phát hiện các giao dịch thẻ tín dụng bất thường để ngăn chặn gian lận, phát hiện lỗi sản xuất hoặc tự động loại bỏ các ngoại lệ khỏi dataset trước khi đưa nó vào một thuật toán học khác. Hệ thống được huấn luyện hầu hết các trường hợp bình thường trong quá trình training, vì vậy nó học cách nhận biết chúng; sau đó, khi nhìn thấy một trường hợp mới, nó có thể cho biết liệu nó trông giống một trường hợp bình thường hay có khả năng là một trường hợp bất thường. Một nhiệm vụ tương tự là phát hiện tính mới: nó nhằm mục đích phát hiện các phiên bản mới trông khác với tất cả các phiên bản trong training set. Điều này đòi hỏi phải có một training set rất “sạch”, không có bất kỳ phiên bản nào mà bạn muốn thuật toán phát hiện. Ví dụ: nếu bạn có hàng nghìn bức ảnh về chó và 1% trong số những bức ảnh này đại diện cho Chihuahua thì thuật toán phát hiện tính mới sẽ không coi các bức ảnh mới về Chihuahua là mới lạ. Mặt khác, các thuật toán phát hiện sự bất thường có thể coi những con chó này rất hiếm và khác biệt so với những con chó khác đến mức chúng có thể classification chúng là những con chó dị thường (không có ý xúc phạm đến Chihuahua).

Cuối cùng, một nhiệm vụ không giám sát phổ biến khác là học luật kết hợp, trong đó mục tiêu là đào sâu vào lượng lớn dữ liệu và khám phá các mối quan hệ thú vị giữa các thuộc tính. Ví dụ: giả sử bạn sở hữu một siêu thị. Việc áp dụng quy tắc kết hợp trên nhật ký bán hàng của bạn có thể tiết lộ rằng những người mua nước sốt thịt nướng và khoai tây chiên cũng có xu hướng mua bánh mì. Vì vậy, bạn có thể muốn đặt các mục này gần nhau.

Semisupervised learning

Vì việc ghi nhãn dữ liệu thường tốn thời gian và chi phí nên bạn sẽ thường có nhiều trường hợp không được gắn nhãn và ít trường hợp được gắn nhãn. Một số thuật toán có thể xử lý dữ liệu được dán nhãn một phần. Điều này được gọi là semisupervised learning.

Một số dịch vụ lưu trữ ảnh, chẳng hạn như Google Photos, là ví dụ điển hình cho điều này. Sau khi bạn upload tất cả ảnh gia đình của mình lên dịch vụ, dịch vụ sẽ tự động nhận ra rằng cùng một người A xuất hiện trong ảnh 1, 5 và 11, trong khi một người B khác xuất hiện trong ảnh 2, 5 và 7. Đây là phần không supervised của thuật toán (clustering). Bây giờ tất cả những gì hệ thống cần là bạn cho nó biết những người này là ai. Chỉ cần thêm một nhãn cho mỗi người và nó có thể đặt tên cho mọi người trong mỗi bức ảnh, điều này rất hữu ích cho việc tìm kiếm ảnh.

Hầu hết các thuật toán semisupervised learning là sự kết hợp của các thuật toán unsupervised và supervised. Ví dụ: deep belief networks (DBNs) dựa trên các thành phần unsupervised được gọi là máy Boltzmann bị hạn chế (RBMs) xếp chồng lên nhau. RBMs được training tuần tự theo cách unsupervised, sau đó toàn bộ hệ thống được tuning bằng kỹ thuật supervised learning.

Đó là khi hệ thống hoạt động hoàn hảo. Trong thực tế, nó thường tạo ra một vài cụm cho mỗi người và đôi khi trộn lẫn hai người trông giống nhau, vì vậy bạn có thể cần cung cấp một vài nhãn cho mỗi người và dọn sạch một số cụm theo cách thủ công.

Reinforcement Learning

Reinforcement Learning là một con thú rất khác. Learning system, được gọi là agent trong bối cảnh này, có thể quan sát môi trường, lựa chọn và thực hiện các hành động và nhận lại phần thưởng (hoặc hình phạt dưới dạng phần thưởng tiêu cực. Sau đó, nó phải tự tìm hiểu đâu là chiến lược tốt nhất, được gọi là chính sách, để nhận được nhiều phần thưởng nhất theo thời gian. Chính sách xác định hành động nào mà agent nên chọn khi ở trong một tình huống nhất định.

Ví dụ: nhiều robot triển khai thuật toán Reinforcement Learning để học cách đi bộ. Chương trình AlphaGo của DeepMind cũng là một ví dụ điển hình về Reinforcement Learning: nó đã gây chú ý vào tháng 5 năm 2017 khi đánh bại nhà vô địch thế giới Ke Jie trong trò chơi cờ vây. Nó học được chính sách chiến thắng của mình bằng cách phân tích hàng triệu ván đấu và sau đó chơi nhiều ván đấu với chính nó. Lưu ý rằng việc học đã bị tắt trong trận đấu với nhà vô địch; AlphaGo chỉ đang áp dụng chính sách mà nó đã học được.

2.1.4.2 Batch Learning và Online Learning

Một tiêu chí khác được sử dụng để classification hệ thống Machine Learning là liệu hệ thống có thể học tăng dần từ luồng dữ liệu đến hay không.

Batch learning

Trong quá trình batch learning, hệ thống không có khả năng học tăng dần: nó phải được training bằng cách sử dụng tất cả dữ liệu có sẵn. Việc này thường tốn nhiều thời gian và tài nguyên máy tính nên thường được thực hiện ngoại tuyến. Đầu tiên, hệ thống được training, sau đó được đưa vào sản xuất và chạy mà không cần học hỏi nữa; nó chỉ áp dụng những gì nó đã học được. Đây được gọi là offline learning.

Nếu bạn muốn hệ thống batch learning biết về dữ liệu mới (chẳng hạn như một loại thư rác mới), bạn cần training phiên bản mới của hệ thống từ đầu trên dataset đầy đủ (không chỉ dữ liệu mới mà còn cả dữ liệu cũ), sau đó dừng hệ thống cũ và thay thế bằng hệ thống mới.

May mắn thay, toàn bộ quá trình training, đánh giá và khởi chạy hệ thống Machine Learning có thể được tự động hóa khá dễ dàng, do đó, ngay cả hệ thống batch learning cũng có thể thích ứng với sự thay đổi. Chỉ cần cập nhật dữ liệu và training phiên bản mới của hệ thống từ đầu thường xuyên nếu cần.

Giải pháp này đơn giản và thường hoạt động tốt, nhưng việc training bằng cách sử dụng bộ dữ liệu đầy đủ có thể mất nhiều giờ, vì vậy bạn thường chỉ training một hệ thống mới sau mỗi 24 giờ hoặc thậm chí chỉ hàng tuần. Nếu hệ thống của bạn cần thích ứng với dữ liệu thay đổi nhanh chóng (ví dụ: để dự đoán giá cổ phiếu), thì bạn cần một giải pháp phản ứng nhanh hơn.

Ngoài ra, việc training trên toàn bộ dữ liệu đòi hỏi nhiều tài nguyên máy tính (CPU, dung lượng bộ nhớ, dung lượng ổ đĩa, I/O ổ đĩa, I/O mạng, v.v.). Nếu bạn có nhiều dữ liệu và bạn tự động hóa hệ thống của mình để training lại từ đầu mỗi ngày, điều đó sẽ khiến bạn tốn rất nhiều tiền. Nếu lượng dữ liệu khổng lồ, thậm chí có thể không sử dụng được thuật toán batch learning.

Cuối cùng, nếu hệ thống của bạn cần có khả năng học tự chủ và có nguồn lực hạn chế (ví dụ: ứng dụng điện thoại thông minh hoặc xe tự hành trên sao Hỏa), thì việc mang theo một lượng lớn dữ liệu training và chiếm nhiều tài nguyên để training hàng giờ mỗi ngày là một sự lựa chọn tuyệt vời.

May mắn thay, một lựa chọn tốt hơn trong tất cả các trường hợp này là sử dụng các thuật toán có khả năng học tăng dần.

Online learning

Trong online learning, bạn training hệ thống tăng dần bằng cách cung cấp cho hệ thống các phiên bản dữ liệu một cách tuần tự, riêng lẻ hoặc theo nhóm nhỏ được gọi là mini-batches. Mỗi bước học đều nhanh và rẻ, do đó hệ thống có thể tìm hiểu về dữ liệu mới một cách nhanh chóng khi nó đến.

Online learning rất phù hợp với các hệ thống nhận dữ liệu dưới dạng luồng liên tục (ví dụ: giá cổ phiếu) và cần thích ứng với sự thay đổi nhanh chóng hoặc tự chủ. Đây cũng là một lựa chọn tốt nếu bạn có tài nguyên máy tính hạn chế: khi online learning system đã tìm hiểu về các phiên bản dữ liệu mới, nó không cần chúng nữa nên bạn có thể loại bỏ chúng (trừ khi bạn muốn có thể quay lại trạng thái trước đó và “phát lại” dữ liệu). Điều này có thể tiết kiệm một lượng lớn không gian.

Các thuật toán online learning cũng có thể được sử dụng để training các hệ thống trên datasets khổng lồ không thể chứa vừa bộ nhớ chính của một máy (điều này được gọi là học ngoài lõi). Thuật toán load một phần dữ liệu, chạy bước training trên dữ liệu đó và lặp lại quy trình cho đến khi chạy trên tất cả dữ liệu.

Học tập bên ngoài thường được thực hiện ngoại tuyến (tức là không phải trên hệ thống trực tiếp), vì vậy online learning có thể là một cái tên khó hiểu. Hãy nghĩ về nó như việc học tập tăng dần.

Một parameter quan trọng của online learning systems là tốc độ thích ứng với việc thay đổi dữ liệu: hệ thống này được gọi là learning rate. Nếu bạn đặt learning rate cao thì hệ thống của bạn sẽ nhanh chóng thích ứng với dữ liệu mới, nhưng nó cũng sẽ có xu hướng nhanh chóng quên dữ liệu cũ (bạn không muốn bộ lọc thư rác chỉ gắn cờ các loại thư rác mới nhất mà nó được hiển thị). Ngược lại, nếu bạn đặt learning rate thấp, hệ thống sẽ có quán tính lớn hơn; nghĩa là, nó sẽ học chậm hơn nhưng cũng sẽ ít nhạy cảm hơn với nhiễu trong dữ liệu mới hoặc với chuỗi các điểm dữ liệu không mang tính đại diện (các điểm ngoại lệ).

Một thách thức lớn với việc online learning là nếu dữ liệu xấu được đưa vào hệ thống, hiệu suất của hệ thống sẽ giảm dần. Nếu đó là một hệ thống trực tiếp, khách hàng của bạn sẽ chú ý. Ví dụ: dữ liệu xấu có thể đến từ cảm biến bị trục trặc trên rô-bốt hoặc từ ai đó gửi thư rác vào công cụ tìm kiếm để cố gắng xếp hạng cao trong kết quả tìm kiếm. Để giảm thiểu rủi ro này, bạn cần giám sát chặt chẽ hệ thống của mình và tắt chế độ học tập ngay lập tức (và có thể trở lại trạng thái hoạt động trước đó) nếu bạn phát hiện thấy hiệu suất giảm sút. Bạn cũng có thể muốn theo dõi dữ liệu đầu vào và phản ứng với dữ liệu bất thường (ví dụ: sử dụng thuật toán phát hiện bất thường).

2.1.4.3 Instance-Based Learning so với Model-Based Learning

Một cách nữa để classification các hệ thống Machine Learning là cách chúng khái quát hóa. Hầu hết các nhiệm vụ của Machine Learning đều là đưa ra dự đoán. Điều này có nghĩa là với một số training examples, hệ thống cần có khả năng đưa ra dự đoán tốt cho các ví dụ (khái quát hóa) mà nó chưa từng thấy trước đây. Có thước đo hiệu suất tốt trên dữ liệu training là tốt nhưng chưa đủ; mục tiêu thực sự là hoạt động tốt trên các phiên bản mới.

Có hai cách tiếp cận chính để khái quát hóa: instance-based learning và model-based learning.

Instance-based learning

Có lẽ hình thức học tập tầm thường nhất chỉ đơn giản là học thuộc lòng. Nếu bạn tạo bộ lọc thư rác theo cách này, nó sẽ chỉ gắn cờ tất cả các email giống hệt với những email đã được user gắn cờ—không phải là giải pháp tồi nhất nhưng chắc chắn không phải là giải pháp tốt nhất.

Thay vì chỉ gắn cờ các email giống hệt với các email spam đã biết, bộ lọc thư rác của bạn có thể được lập trình để gắn cờ các email rất giống với các email spam đã biết. Điều này đòi hỏi thước đo về sự giống nhau giữa hai email. Một thước đo về sự giống nhau (rất cơ bản) giữa hai email có thể là đếm số từ chung giữa chúng. Hệ thống sẽ gắn cờ một email là thư rác nếu nó có nhiều từ chung với một email spam đã biết.

Đây được gọi là instance-based learning: hệ thống học thuộc lòng các ví dụ, sau đó khái quát hóa các trường hợp mới bằng cách sử dụng thước đo độ tương tự để so sánh chúng với các ví dụ đã học (hoặc một tập hợp con của chúng). Ví dụ, phiên bản mới sẽ được phân loại là một hình tam giác vì phần lớn các phiên bản tương tự nhất đều thuộc về lớp đó.

Model-based learning

Một cách khác để khái quát hóa từ một tập hợp các ví dụ là xây dựng model của các ví dụ này và sau đó sử dụng model đó để đưa ra dự đoán. Đây được gọi là model-based learning.

Ví dụ: giả sử bạn muốn biết liệu tiền có làm mọi người hạnh phúc hay không, vì vậy bạn tải xuống dữ liệu Chỉ số cuộc sống tốt hơn từ trang web của OECD và số liệu thống kê về tổng sản phẩm quốc nội (GDP) bình quân đầu người từ trang web của IMF. Sau đó, bạn nối các bảng và sắp xếp theo GDP bình quân đầu người. Bảng 1-1 cho thấy một đoạn trích về những gì bạn nhận được.

Quốc gia GDP bình quân đầu người (USD) Sự hài lòng về cuộc sống

Hungary 12.240 4,9

Hàn Quốc 27.195 5,8

Pháp 37.675 6,5

Úc 50.962 7,3

Hoa Kỳ 55.805 7,2

Hãy vẽ biểu đồ dữ liệu cho các quốc gia này.

Có vẻ như có một xu hướng ở đây! Mặc dù dữ liệu không ổn định (tức là một phần ngẫu nhiên), nhưng có vẻ như mức độ hài lòng về cuộc sống tăng lên ít nhiều tuyến tính khi GDP bình quân đầu người của quốc gia tăng lên. Vì vậy, bạn quyết định model sự hài lòng trong cuộc sống như một hàm tuyến tính của GDP bình quân đầu người. Bước này được gọi là lựa chọn model: bạn đã chọn một model tuyến tính có mức độ hài lòng với cuộc sống chỉ với một thuộc tính, GDP bình quân đầu người.

Phương trình 1-1. Model tuyến tính đơn giản $\text{life_satisfaction} = \text{theta}_0 + \text{theta}_1 \times \text{GDP_per_capita}$

Theo quy ước, chữ cái Hy Lạp theta (theta) thường được sử dụng để biểu thị model parameters.

Model này có hai model parameters, theta_0 và theta_1 . Bằng cách điều chỉnh các parameters này, bạn có thể làm cho model của bạn biểu thị bất kỳ hàm tuyến tính nào

Trước khi có thể sử dụng model, bạn cần xác định các giá trị parameter theta_0 và theta_1 . Làm cách nào bạn có thể biết giá trị nào sẽ giúp model của bạn hoạt động tốt nhất? Để trả lời câu hỏi này, bạn cần chỉ định thước đo hiệu suất. Bạn có thể xác định hàm tiện ích (hoặc hàm phù hợp) để đo lường mức độ tốt của model của bạn hoặc bạn có

thể xác định hàm chi phí để đo lường mức độ tệ hại của nó. Đối với các bài toán Linear Regression, mọi người thường sử dụng hàm chi phí để đo khoảng cách giữa các dự đoán của model tuyến tính và training examples; mục tiêu là giảm thiểu khoảng cách này.

Đây là lúc thuật toán Linear Regression xuất hiện: bạn cung cấp cho nó training examples của mình và nó tìm thấy parameters làm cho model tuyến tính phù hợp nhất với dữ liệu của bạn. Điều này được gọi là training model. Trong trường hợp của chúng ta, thuật toán nhận thấy rằng các giá trị parameter tối ưu là $\theta_0 = 4,85$ và $\theta_1 = 4,91 \times 10^{-5}$.

Điều gây nhầm lẫn là cùng một từ “model” có thể đề cập đến một loại model (ví dụ: Linear Regression), đến kiến trúc model được chỉ định đầy đủ (ví dụ: Linear Regression với một đầu vào và một đầu ra) hoặc đến model đã được training cuối cùng sẵn sàng để sử dụng cho các dự đoán (ví dụ: Regression tuyến tính với một đầu vào và một đầu ra, sử dụng $\theta_0 = 4,85$ và $\theta_1 = 4,91 \times 10^{-5}$). Lựa chọn Model bao gồm việc chọn loại model và chỉ định đầy đủ kiến trúc của nó. Training model có nghĩa là chạy thuật toán để tìm model parameters phù hợp nhất với dữ liệu training (và hy vọng đưa ra dự đoán tốt về dữ liệu mới).